

MLOps for a malware domain name detector

Project in DD2482

Vilhelm Prytz vpрызt@kth.se
Filip Dimitrijevic filipdi@kth.se

7th October 2026

1 Introduction

Malware uses DGAs (domain generation algorithms) to find its command-and-control servers. Our goal is to detect such domains with a logistic regression classifier.

The focus of this project is not the model but the MLOps pipeline around it, as outlined in Colin Raffel’s article.¹

In our pipeline, every model change is tested automatically and compared with the latest release, then released with a semantic version and deployed. The benign domains come from the Tranco list,² and the DGA domains come from generators for known malware families.

2 Automated build and testing

The project uses a CI pipeline in GitHub Actions, `.github/workflows/ci.yml`. It runs `ruff` and `Black` for linting and formatting, and `pytest` for the unit tests. It also builds the data, runs the data tests, trains the model, and then runs our gate against the latest release’s `metrics.json`. The gate checks our key metrics, the false-negative rate (FNR) and the false-positive rate (FPR). If a metric breaks its limit in `config.toml`, or is worse than in the latest release by more than a small tolerance, the gate fails the job, so a worse model cannot make it to production. This applies DevOps principles to ML.

Furthermore, a container image with the tested model is built and smoke-tested, and on merges to `main` it is pushed to GitHub Container Registry. We have also added a workflow that runs `Semgrep`, an open-source static analysis (SAST) tool that scans code for security vulnerabilities, bugs, and compliance issues.

3 Automated deployment or delivery

Every merge to `main` automatically creates a new GitHub release (with `model.skops` and `metrics.json`), which is continuous delivery. It also deploys the new version of the web app to a Raspberry Pi at home, which is continuous deployment. The deploy job joins our Tailscale network with an ephemeral auth key and connects to the Pi over SSH. There, a script pulls the new version and starts it. It then checks that the `/health` endpoint passes, which only happens if the new model correctly classifies a known benign domain (`google.com`) and a known necurs domain. The endpoint also reports the version and the git commit. If the check fails, the previous version is started again automatically (automatic rollback).

Versioning is automatic: before merging a PR, we can add a `release:minor` or `release:major` label to indicate whether the change should lead to a minor or a major release. Without a label, the patch version is bumped. This creates a feedback loop, since the new release’s `metrics.json` becomes the baseline for all subsequent PRs.

4 Infrastructure configuration

All infrastructure is defined as code in the repository. This includes the CI/CD configuration and an Ansible playbook that turns a fresh Raspberry Pi OS installation into a production host (Docker, Tailscale and a deployment user). The Dockerfile, the workflows and `config.toml` (model parameters and gate limits) are also versioned and declarative.

¹<https://cacm.acm.org/opinion/building-machine-learning-models-like-open-source-software/>

²<https://tranco-list.eu/>

5 Development platform

We use GitHub for everything except the deployment, which runs on a Raspberry Pi. The repository, pull requests and a ruleset on `main` (which requires the GitHub Actions jobs to pass and an approving review before merging) are all on GitHub. We also use GitHub Actions, GitHub Container Registry and GitHub Releases. The benefit is having everything on one development platform.

6 Quality or security automation

Semgrep runs on every pull request. We have also configured Dependabot for the Python packages, GitHub Actions and the Docker base image. Dependabot runs weekly, and Dependabot security updates are also enabled.

7 Documented use of AI-assisted tools

We used Claude Code (Anthropic) to help with the boilerplate for the DGA generator; the CI/CD and infrastructure are our own work. GitHub Copilot code review runs on every pull request through the repository ruleset. We also used AI tools to proofread this report for spelling mistakes and poor wording, and to generate the $\text{\LaTeX}$ code for the figure in Section 9.

8 Project repository

The repository³ contains a fully functional implementation, including all configuration, code and documentation needed to run the system.

9 How components interact

See Figure 1. When a PR is opened, `test` and `model` start in parallel. The `model` job trains the model and compares its metrics with those of the latest release. If both jobs pass, `image` builds the container image with that model and smoke-tests it.

10 Key design decisions

We treat the model as a compiled binary, like any other software artifact, which applies DevOps principles to ML. The model is trained once in CI, and the gate stops it if its metrics get worse. The baseline comes from the latest release, as calculated by CI to ensure the metrics are reproducible.

We use one platform for everything, so the required checks enforce the gate in the same place as code review, with one permission model and no integrations between services to maintain. We also have a Makefile that CI runs and that can be run locally, so the same commands run in CI and on our machines.

To make the environment reproducible, we set up the Raspberry Pi with an Ansible playbook (IaC), so it can be recreated from a fresh OS without manual configuration drift. CI reaches the Pi over Tailscale, so no ports are open on our home network. Finally, the release type is chosen with a PR label, since whether a change is major or minor is a human judgement, and the label makes that decision visible in code review.

11 Limitations and trade-offs

One limitation is the model itself, which is deliberately simple. The deployment is also simplified: the web app runs on a single Raspberry Pi with no staging environment. It is also only deployed when a new release is created, so PRs are not deployed the way they would be with review deployments.

The pipeline itself also has a few trade-offs: the gate's tolerance is hand-picked, and since one test domain moves a metric by 0.0005, it could block a legitimate change or let a small regression through. Also, having everything on GitHub comes at the price of lock-in. Furthermore, every merge creates a release, even for documentation-only changes, which is also a trade-off we chose for simplicity.

³<https://github.com/vilhelmprytz/devops-project>

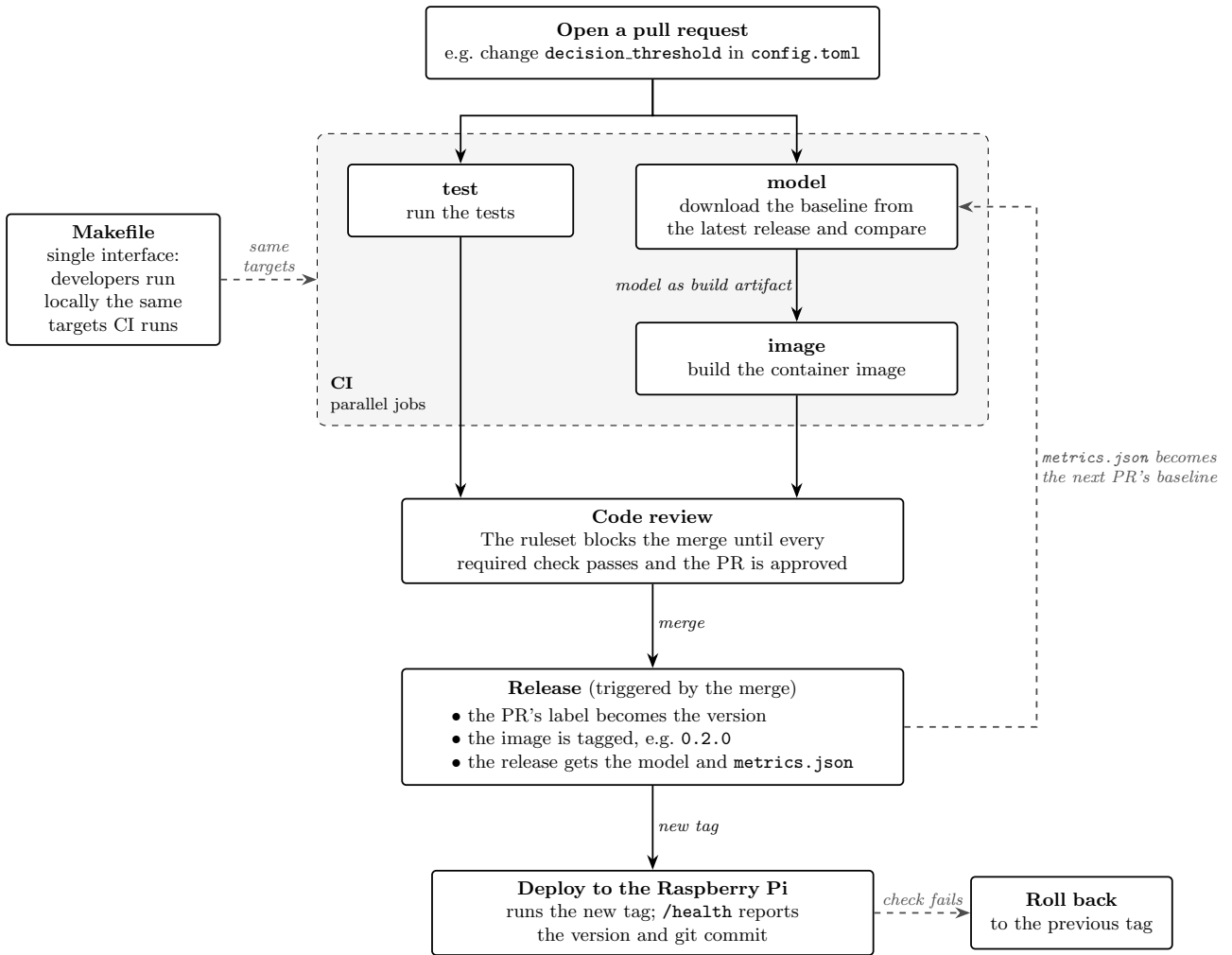


Figure 1: The path of one change, from pull request to deployment.

12 Future work

Future work would be to improve the model. We also think review deployments would make sense in this project, since they would let PR reviewers try the model through a deployed URL of the web app. They would also ensure that every PR is deployable, since each PR would be deployed to a separate environment. Furthermore, separate staging and production environments would make sense too.